

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

2026-27
ODD SEMESTER

**INFORMATION ASSURANCE AND SECURITY
(24CS3105)**

ALM – PROTOTYPE REVIEW

**SECURE CLIENT–SERVER COMMUNICATION
USING CAESAR, PLAYFAIR AND
SIMPLIFIED DES CIPHERS**

submitted by

Trinadh Koundinya V 2420090121

Cluster: 4 Section: 11

Course Instructor
Dr. Jagan Mohan Reddy Danda
Associate Professor



KONERU LAKSHMAIAH EDUCATION FOUNDATION, BOWRAMPET

Hyderabad-500043, Telangana, India

2026-27

Abstract

This report documents the design, implementation and evaluation of a prototype client-server application that provides application-layer confidentiality for messages and bulk files using three ciphers of increasing sophistication: the Caesar substitution cipher, the Playfair digraph cipher and a byte-wise Simplified Data Encryption Standard (S-DES). The prototype is written in Python and consists of four components: a reusable cryptographic module (`ciphers.py`), a TCP server (`server.py`), an interactive TCP client (`client.py`) and a sample-data generator (`make_files.py`). The client encrypts user input or a 1 MB binary file, prefixes it with a four-byte length field and a JSON header describing the mode, algorithm and key, and transmits the frame over a TCP socket to the server on port 9000. The server reconstructs the frame, decrypts the payload, displays both ciphertext and recovered plaintext, and answers with an encrypted reply or an S-DES-encrypted 10 KB file. Functional testing confirmed loss-free round trips for all three ciphers: the 1 MB file recovered by the server was byte-identical to the original, as was the 10 KB file returned to the client. Measured throughput of the educational S-DES implementation was approximately 0.13 MB/s (7.58 s to encrypt and 7.57 s to decrypt 1 MB), which quantifies the cost of a pure-Python, block-at-a-time design. The testing chapter also records the limits observed in practice: Caesar offers 25 effective keys, S-DES offers a 10-bit key space of 1,024 keys, encryption is performed in Electronic Codebook mode so identical plaintext bytes produce identical ciphertext bytes, no integrity or authentication mechanism is present, and the session key travels in cleartext inside the JSON header. The prototype therefore achieves obfuscation suitable for teaching rather than real confidentiality.

Contents

List of Tables	4
List of Figures	6
1 Introduction	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Objectives	2
1.4 Scope and Assumptions	2
1.5 Development Environment	3
2 System Design	4
2.1 Architecture Overview	4
2.2 Module Responsibilities	5
2.3 Message Frame Format	5
2.4 Control Flow	6
3 Cryptographic Algorithms	7
3.1 Caesar Cipher	7
3.2 Playfair Cipher	8

3.3	Simplified DES	9
3.4	Key Material	10
4	Implementation	11
4.1	Cipher Module	11
4.2	Client	12
4.3	Server	12
4.4	Bulk File Handling	13
4.5	Execution Procedure	13
5	Testing and Results	14
5.1	Methodology	14
5.2	Functional Test Cases	14
5.3	Console Transcripts	14
5.4	Performance	15
5.5	Observations and Known Defects	16

List of Tables

1.1	Prototype implementation and test environment	3
2.1	Source files and their responsibilities	5
2.2	Fields carried in the JSON header	6
3.1	Permutation tables used by the S-DES implementation	10
3.2	Key formats and effective key space per algorithm	10
5.1	Functional test cases and observed results	15
5.2	Measured S-DES performance on the test machine	16

List of Figures

2.1	Component architecture of the client–server cipher prototype; both end-points share one cipher module and communicate over a TCP socket on port 9000.	4
2.2	Application-layer frame: four-byte length prefix, JSON header and ciphertext payload.	5

Chapter 1

Introduction

1.1 Background

Confidentiality is the first pillar of the CIA triad and the historical starting point of cryptography. Classical ciphers such as Caesar and Playfair illustrate substitution and digraph techniques, while Simplified DES (S-DES) is a scaled-down teaching version of the Feistel block-cipher structure used by DES and, in spirit, by AES. Studying all three inside a single working network application makes the transition from paper exercises to real message and file protection concrete: the same plaintext must be encoded, framed, transmitted over a socket, received, decrypted and verified byte-for-byte.

1.2 Problem Statement

A plain TCP socket transfers data in cleartext, so any party on the network path can read messages and files. The problem addressed by this prototype is therefore: *how can a client and a server exchange text messages and bulk binary files over TCP such that only the endpoints holding the shared key can read the content, while the mechanism*

itself remains simple enough to inspect and reason about?

1.3 Objectives

1. Implement Caesar, Playfair and byte-wise S-DES encryption and decryption as a reusable Python module.
2. Build a TCP server that accepts an encrypted frame, decrypts it, and displays the ciphertext alongside the recovered plaintext.
3. Build an interactive client that lets the user choose the algorithm and key, encrypt either a typed message or a 1 MB binary file, and decrypt the server's reply.
4. Verify correctness by comparing the recovered file with the original byte-for-byte, and measure the throughput of the S-DES implementation.
5. Analyse the residual security weaknesses of the prototype and recommend defensible alternatives.

1.4 Scope and Assumptions

The prototype is an educational artefact. The shared key is agreed out of band (the user types it into both endpoints) and is carried inside the frame header for convenience; no key exchange, certificate validation, integrity check or replay protection is implemented. S-DES operates one byte at a time in Electronic Codebook (ECB) mode. Caesar and Playfair operate on text only, and Playfair additionally discards non-alphabetic characters. Testing was carried out on the loopback interface with one client connection at a time, because the server handles connections sequentially.

1.5 Development Environment

Table 1.1: Prototype implementation and test environment

Item	Value
Language	Python 3.14.3 (standard library only)
Modules used	<code>socket</code> , <code>json</code> , <code>os</code> , <code>math</code>
Transport	TCP, stream sockets (<code>AF_INET</code> , <code>SOCK_STREAM</code>)
Server bind address	<code>0.0.0.0</code> , port 9000
Client target	<code>127.0.0.1</code> , port 9000
Ciphers	Caesar, Playfair, S-DES (byte-wise, ECB)
Test data	<code>sample_1mb.bin</code> (1,048,576 B), <code>sample_10kb.bin</code> (10,240 B)
Source repository	https://github.com/Trinadh-31/IAS

Chapter 2

System Design

2.1 Architecture Overview

The prototype follows a three-layer split: a cipher library, two network endpoints, and a sample-data generator. Both endpoints import the same cipher functions, so encryption and decryption are guaranteed to be symmetric. Figure 2.1 shows the arrangement and the direction of data flow.

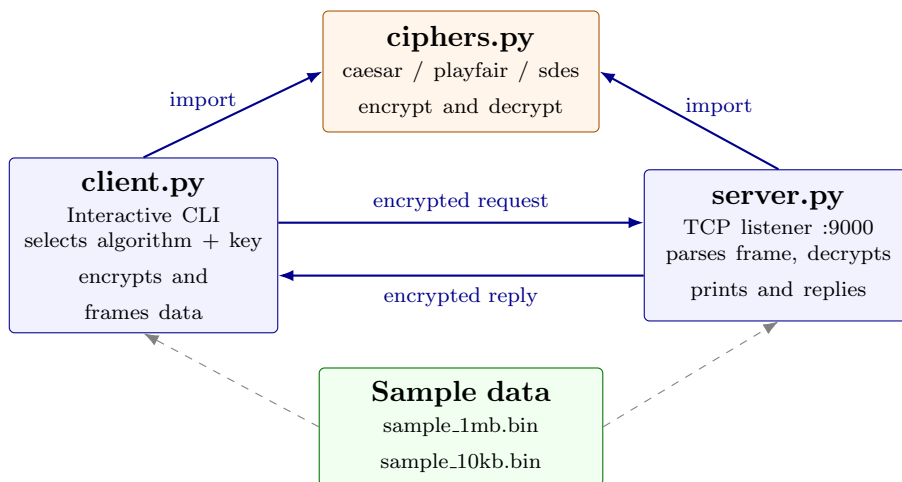


Figure 2.1: Component architecture of the client-server cipher prototype; both endpoints share one cipher module and communicate over a TCP socket on port 9000.

2.2 Module Responsibilities

Table 2.1 lists each source file and the responsibility it owns. Keeping the ciphers in one module avoids duplicated key-schedule logic and makes the algorithms independently testable.

Table 2.1: Source files and their responsibilities

File	Responsibility
<code>ciphers.py</code>	Caesar, Playfair and S-DES primitives; self-test in <code>__main__</code>
<code>server.py</code>	Listener, frame parsing, decryption, display, encrypted reply
<code>client.py</code>	Menu-driven UI, key entry, encryption, framing, reply handling
<code>make_files.py</code>	Generates the 1 MB and 10 KB random sample files

2.3 Message Frame Format

Because TCP is a byte stream and not a message-oriented protocol, the prototype defines its own application-layer framing, shown in Figure 2.2. A four-byte big-endian length prefix tells the peer how many bytes of JSON header follow; the header in turn declares `payload_size`, so the receiver knows exactly how many ciphertext bytes to read. The helper `recv_all()` loops until the requested number of bytes has arrived, which prevents the short-read bug that naive `recv()` calls suffer on large transfers.

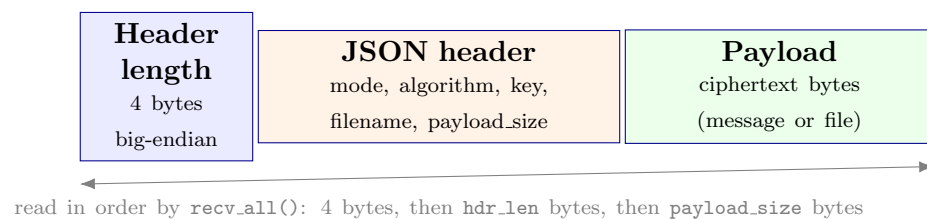


Figure 2.2: Application-layer frame: four-byte length prefix, JSON header and ciphertext payload.

Table 2.2: Fields carried in the JSON header

Field	Example	Purpose
mode	message / file	Selects the receiver's handling path
algorithm	caesar / playfair / sdes	Selects the decryption routine
key	3 / SECURE / 1010000010	Shared key for that algorithm
filename	sample_1mb.bin	Output name for file mode
payload_size	1048576	Exact ciphertext length in bytes

2.4 Control Flow

The client presents a menu with three actions: send a message, send the 1 MB file using S-DES, or quit. For a message the user picks one of the three algorithms and supplies the matching key; the client prints the ciphertext, opens a fresh connection, transmits the frame and waits for the reply. The server runs an accept loop: for every connection it reads one frame, decrypts it, prints both representations, and then responds. For Caesar and Playfair the response is an encrypted acknowledgement string; for S-DES the response is an encrypted 10 KB file, which exercises the bulk path in the opposite direction.

Chapter 3

Cryptographic Algorithms

3.1 Caesar Cipher

The Caesar cipher shifts every alphabetic character by a fixed integer, wrapping within its case and leaving digits, spaces and punctuation untouched. Decryption reuses the encryption routine with the additive inverse of the shift, $(-k) \bmod 26$, which halves the code that has to be trusted.

Algorithm 1: Caesar encryption as implemented in `encrypt_caesar()`

Input: plaintext P , shift k **Output:** ciphertext C $C \leftarrow \varepsilon;$ **foreach** *character* $c \in P$ **do** **if** c *is lowercase* **then** append $\text{chr}((\text{ord}(c) - \text{ord}(\text{'a'}) + k) \bmod 26 + \text{ord}(\text{'a'}))$ to C ; **else if** c *is uppercase* **then** append $\text{chr}((\text{ord}(c) - \text{ord}(\text{'A'}) + k) \bmod 26 + \text{ord}(\text{'A'}))$ to C ; **else** append c unchanged to C ; **end****end****return** C ;

3.2 Playfair Cipher

Playfair encrypts digraphs using a 5×5 key matrix. The implementation normalises the text to uppercase letters, maps J to I, inserts X between repeated letters of a pair, and pads an odd-length string with a trailing X. Three substitution rules then apply to each pair: same row shifts right, same column shifts down, otherwise the letters are replaced by the opposite corners of the rectangle they define.

Algorithm 2: Playfair encryption as implemented in `encrypt_playfair()`

Input: plaintext P , keyword K
Output: ciphertext C
 $T \leftarrow$ normalise and digraph-pad P (uppercase, $J \rightarrow I$, insert X);

 $M \leftarrow 5 \times 5$ matrix from unique letters of K , then remaining alphabet without J ;

for $i \leftarrow 0$ **to** $|T| - 1$ **step 2 do**

 $(r_a, c_a) \leftarrow \text{pos}(T_i); \quad (r_b, c_b) \leftarrow \text{pos}(T_{i+1});$

 if $r_a = r_b$ **then**

 append $M[r_a][(c_a+1) \bmod 5]$ and $M[r_b][(c_b+1) \bmod 5]$;

 else if $c_a = c_b$ **then**

 append $M[(r_a+1) \bmod 5][c_a]$ and $M[(r_b+1) \bmod 5][c_b]$;

 else

 append $M[r_a][c_b]$ and $M[r_b][c_a]$;

 end
end
return C ;

3.3 Simplified DES

S-DES is a two-round Feistel cipher with an 8-bit block and a 10-bit key. The key schedule permutes the key with P10, splits it into two halves, and applies circular left shifts of one and two positions before compressing each result with P8 to produce subkeys K_1 and K_2 . Each round applies the function f_K : expand/permute the right half with EP, XOR with the subkey, substitute through S-boxes S_0 and S_1 , permute with P4, and XOR into the left half. The block is swapped between rounds, and the inverse initial permutation ends the process. Decryption is identical with the subkeys

applied in reverse order.

Table 3.1: Permutation tables used by the S-DES implementation

Table	Values
P10	3, 5, 2, 7, 4, 10, 1, 9, 8, 6
P8	6, 3, 7, 4, 8, 5, 10, 9
IP	2, 6, 3, 1, 4, 8, 5, 7
IP^{-1}	4, 1, 3, 5, 7, 2, 8, 6
EP	4, 1, 2, 3, 2, 3, 4, 1
P4	2, 4, 3, 1

Algorithm 3: S-DES single-block encryption (`sdes_encrypt_block()`)

Input: plaintext byte b , 10-bit key K

Output: ciphertext byte b'

```

( $K_1, K_2$ )  $\leftarrow$  generate_subkeys( $K$ ) ;                               // P10, shifts, P8
 $x \leftarrow IP(bits(b))$ ;
 $x \leftarrow f_K(x, K_1)$  ;                                           // EP, XOR,  $S_0/S_1$ , P4
 $x \leftarrow swap(x)$  ;                                              // exchange 4-bit halves
 $x \leftarrow f_K(x, K_2)$ ;
 $b' \leftarrow byte(IP^{-1}(x))$ ;
return  $b'$ ;

```

3.4 Key Material

Table 3.2 summarises the key formats and the size of the resulting search space, which determines how easily each cipher can be brute-forced.

Table 3.2: Key formats and effective key space per algorithm

Algorithm	Key format	Effective key space
Caesar	integer shift, e.g. 3	25 useful keys
Playfair	keyword, e.g. SECURE	$25! \approx 1.55 \times 10^{25}$ matrices, far fewer in practice
S-DES	10-bit string, e.g. 1010000010	$2^{10} = 1,024$ keys

Chapter 4

Implementation

4.1 Cipher Module

The cipher module exposes symmetric pairs of functions and a bytes-level wrapper for S-DES that iterates over the input one byte at a time. The listing below shows the key schedule and the bulk wrapper.

```
1 def _generate_sdes_subkeys(key10):
2     p10 = [3,5,2,7,4,10,1,9,8,6]
3     p8  = [6,3,7,4,8,5,10,9]
4     k = _permute(key10, p10)
5     left, right = k[:5], k[5:]
6     left, right = _left_shift(left,1), _left_shift(right,1)
7     k1 = _permute(left+right, p8)
8     left, right = _left_shift(left,2), _left_shift(right,2)
9     k2 = _permute(left+right, p8)
10    return k1, k2
11
12 def sdes_encrypt_bytes(data: bytes, key10: str) -> bytes:
13     if len(key10) != 10:
14         raise ValueError('SDES key must be 10 bits')
15     key_bits = [int(c) for c in key10]
16     out = bytearray()
17     for b in data:
18         out.append(sdes_encrypt_block(b, key_bits))
```

```
19 return bytes(out)
```

Listing 4.1: S-DES key schedule and bulk encryption (`ciphers.py`)

4.2 Client

The client builds the frame and writes it in three ordered sends, then reads the reply with the same discipline.

```
1 def send_header_and_payload(conn, header: dict, payload: bytes):
2     hdr_b = json.dumps(header).encode()
3     conn.send(len(hdr_b).to_bytes(4, 'big'))
4     conn.send(hdr_b)
5     if payload:
6         conn.send(payload)
7
8 header = {'mode': 'message', 'algorithm': 'sdes', 'key': key,
9          'payload_size': len(payload)}
```

Listing 4.2: Framing and transmission on the client (`client.py`)

4.3 Server

On the server side `recv_all()` guarantees complete reads, and the handler dispatches on mode and algorithm.

```
1 def recv_all(conn, n):
2     buf = b''
3     while len(buf) < n:
4         chunk = conn.recv(n - len(buf))
5         if not chunk:
6             raise ConnectionError('socket closed')
7         buf += chunk
8     return buf
9
10 hdr_len = int.from_bytes(recv_all(conn, 4), 'big')
11 hdr = json.loads(recv_all(conn, hdr_len).decode())
12 payload = recv_all(conn, hdr.get('payload_size', 0))
```

Listing 4.3: Complete reads and frame parsing on the server (`server.py`)

4.4 Bulk File Handling

In file mode the client reads `sample_1mb.bin`, encrypts the whole buffer with S-DES and sends it as a single payload. The server stores the ciphertext as `received_sample_1mb.bin`, writes the decrypted result as `decrypted_sample_1mb.bin`, and then encrypts and returns `sample_10kb.bin`. The client decrypts that response into `server_sent_sample_10kb.bin`. This produces four artefacts that can be compared with the originals to prove that no bytes were lost or altered.

4.5 Execution Procedure

```
1 python make_files.py      # generate sample_1mb.bin and
    sample_10kb.bin
2 python server.py          # terminal 1: listener on port 9000
3 python client.py          # terminal 2: interactive menu
4 # SDES key must be exactly 10 bits, e.g. 1010000010
```

Listing 4.4: Running the prototype

Chapter 5

Testing and Results

5.1 Methodology

Each cipher was exercised in message mode, and S-DES was additionally exercised in file mode with the 1 MB sample. Correctness was judged on two criteria: the plaintext printed by the receiver had to match what was sent, and the decrypted file on disk had to be byte-identical to the source file. Timings were taken around the bulk encryption and decryption calls on the same machine that hosted both endpoints.

5.2 Functional Test Cases

Table 5.1 lists the executed cases with the values actually observed in the terminals.

5.3 Console Transcripts

The following transcript was captured on the server while the Caesar and S-DES message tests ran.

Table 5.1: Functional test cases and observed results

#	Case	Key	Observed output	Result
1	Caesar message	3	Lqirupdwlrq Dvvxudqfh dqg Vhfxulwb	Pass
2	Caesar reply decrypt	3	Server reply: received your message	Pass
3	Playfair message	SECURE	ISKYIQEWUMKC	Pass
4	Playfair reply decrypt	SECURE	SERVERREPLYRECEIVEDYOURMESSAGE	Pass
5	S-DES message HELLO	1010000010	ciphertext e050a5a547, server printed b'HELLO'	Pass
6	S-DES 1 MB file transfer	1010000010	decrypted_sample_1mb.bin identical, 1,048,576 B	Pass
7	S-DES 10 KB reply file	1010000010	server_sent_sample_10kb.bin identical, 10,240 B	Pass
8	Playfair round trip of HELLO WORLD	SECURE	recovered HELXLOWORLDX	Partial
9	Invalid S-DES key length	101	ValueError: SDES key must be 10 bits	Pass

```

1 Starting server on 0.0.0.0 9000
2 Client connected ('127.0.0.1', 55270)
3 Received mode=message algorithm=caesar key=3 size=34
4 Ciphertext (client->server): Lqirupdwlrq Dvvxudqfh dqg Vhfxulwb
5 Decrypted plaintext: Information Assurance and Security
6 Sent reply ciphertext and plaintext shown above (server side)
7 Client connected ('127.0.0.1', 46710)
8 Received mode=message algorithm=sdes key=1010000010 size=5
9 Received SDES ciphertext (hex, first 200 bytes): e050a5a547
10 Decrypted plaintext (first 200 bytes): b'HELLO '
11 Server will send SDES-encrypted 10KB file back to client
12 Sent encrypted 10KB file to client

```

Listing 5.1: Server output during message tests

```

1 Enter 10-bit SDES key to use: 1010000010
2 Encrypted 1MB file preview (hex, first 200 bytes):
   d7207a0cefabd6c927ac9d5b...
3 Received file header from server: {'mode': 'file', 'algorithm':
   'sdes',
4   'key': '1010000010', 'filename': 'sample_10kb.bin', '
   payload_size': 10240}
5 Saved decrypted file from server to server_sent_sample_10kb.bin

```

Listing 5.2: Client output during the 1 MB S-DES file transfer

5.4 Performance

Table 5.2 reports the measured cost of the byte-wise S-DES implementation. The throughput of roughly 0.13 MB/s is a direct consequence of processing eight bits per Python function call; a table-driven or block-based implementation, or a call into a

compiled library, would raise this by orders of magnitude. Caesar and Playfair were applied only to short messages, where the cost was not measurable at this resolution.

Table 5.2: Measured S-DES performance on the test machine

Payload	Encrypt	Decrypt	Throughput
1 MB (1,048,576 B)	7.58 s	7.57 s	≈ 0.13 MB/s
10 KB (10,240 B)	0.073 s	0.073 s	≈ 0.13 MB/s
5 B message (HELLO)	< 0.001 s	< 0.001 s	not measurable

5.5 Observations and Known Defects

1. **Playfair is not loss-free.** Because the preparation step strips spaces and inserts filler letters, `HELLO WORLD` decrypts to `HELXLOWORLDX`. This is inherent to Playfair, but it means the cipher cannot be used for binary or exact-text payloads.
2. **Asymmetric reply in S-DES message mode.** When a message is sent with S-DES the server answers with a `file` frame, while the client only prints `Server sent a file back; use receive logic`. The reply path for that combination is unfinished.
3. **Key echoed in the header.** Every frame carries the key in cleartext, so an observer on the path recovers the plaintext trivially.
4. **Single-threaded server.** Connections are handled one at a time, so a long 1 MB transfer blocks all other clients.
5. **No padding or ECB alternative.** S-DES encrypts each byte independently, so repeated bytes are visible in the ciphertext.